

PA2 - Matrix Transformations and Perspective

Tair Asset | AIB-2401 | Student ID 242458

E1 - Order of transformations

Prediction: The solid uses $M = R_x(2\pi t/14) \cdot T(0,2.5,0) \cdot R_y(2t) \cdot S(s)$, where $s = 0.65 + 0.15 \sin(2\pi t/3)$. Swapping R_x and T should stop the orbit. Swapping T and R_y should leave this variant unchanged.

Result: At $t=3.5$ s, the original centre is $(0,0,2.5)$. With $M = T \cdot R_x \cdot R_y \cdot S$, it stays at $(0,2.5,0)$, although the solid still rotates. With $M = R_x \cdot R_y \cdot T \cdot S$, the motion looks the same as the original (Figures 1-3).

Explanation: glmMatrix multiplies on the right, so the last transformation acts first. $R_x \cdot T$ rotates the translated centre around the origin. $T \cdot R_x$ rotates the solid before placing its centre at a fixed position. Translation along y commutes with rotation around y . This is a special case, not a general rule.

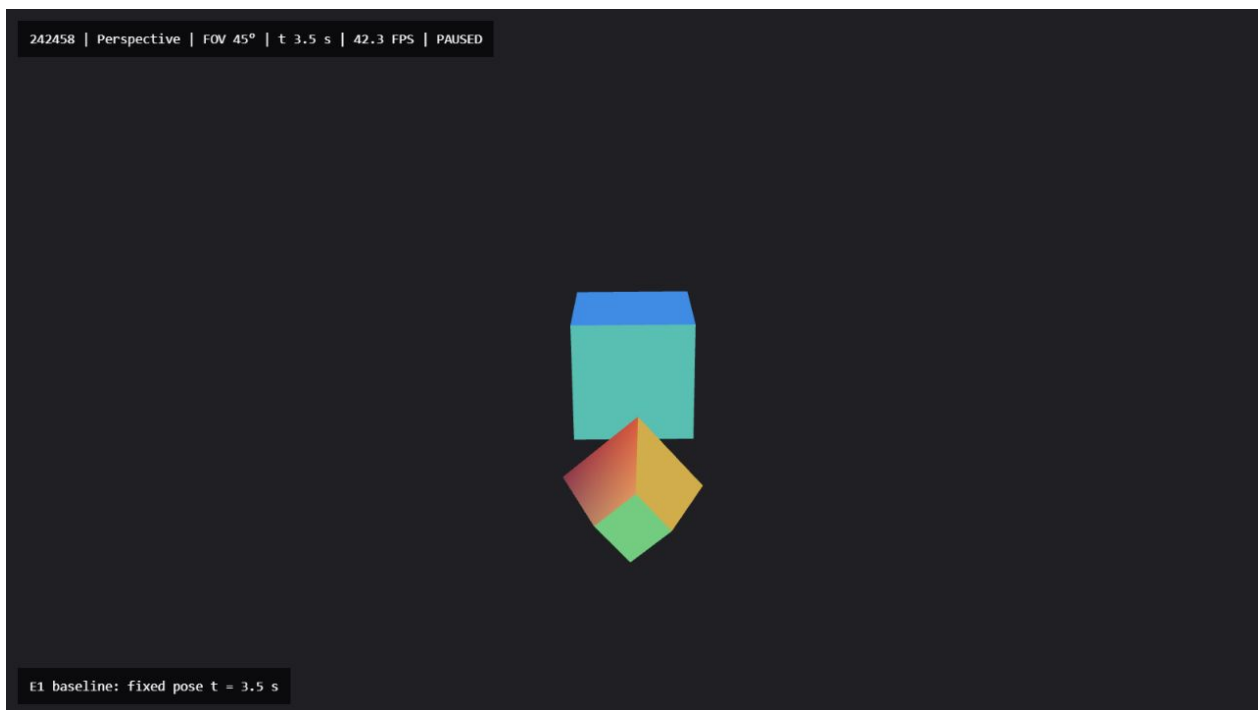


Figure 1. E1 baseline, paused at $t=3.5$ s.

E1 - Evidence of both swaps

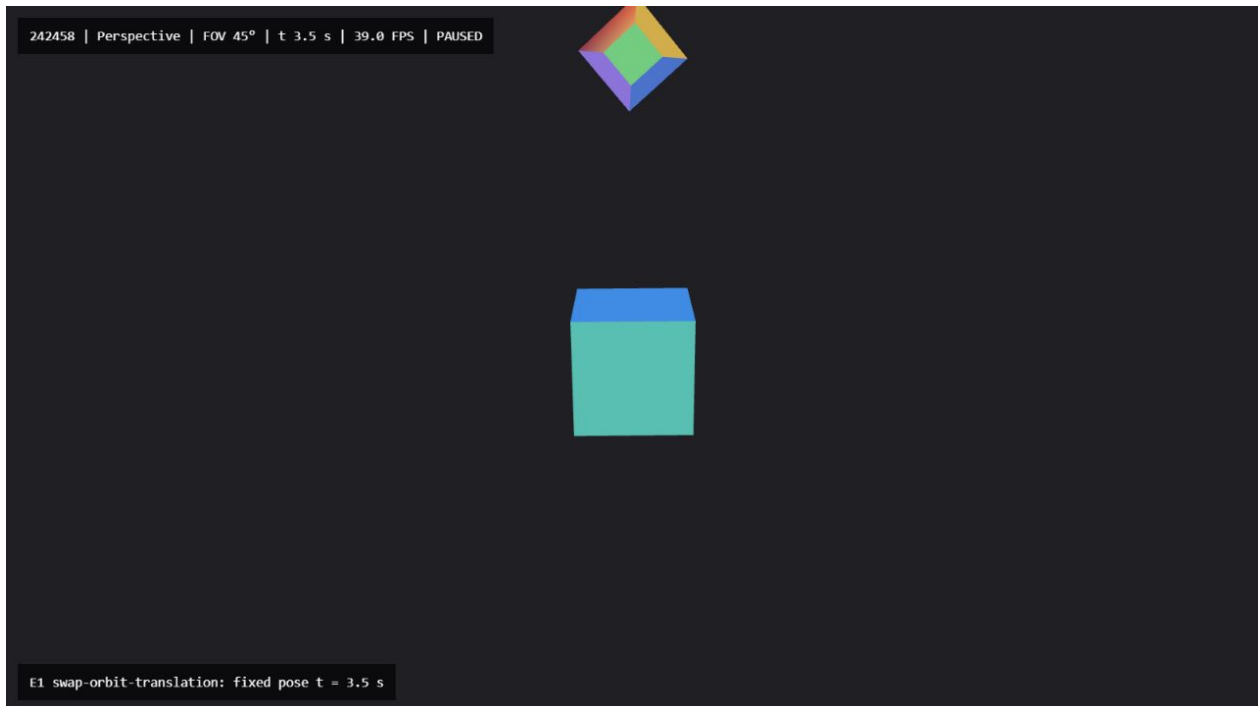


Figure 2. Orbit/translation swapped: the centre stays at (0,2.5,0).

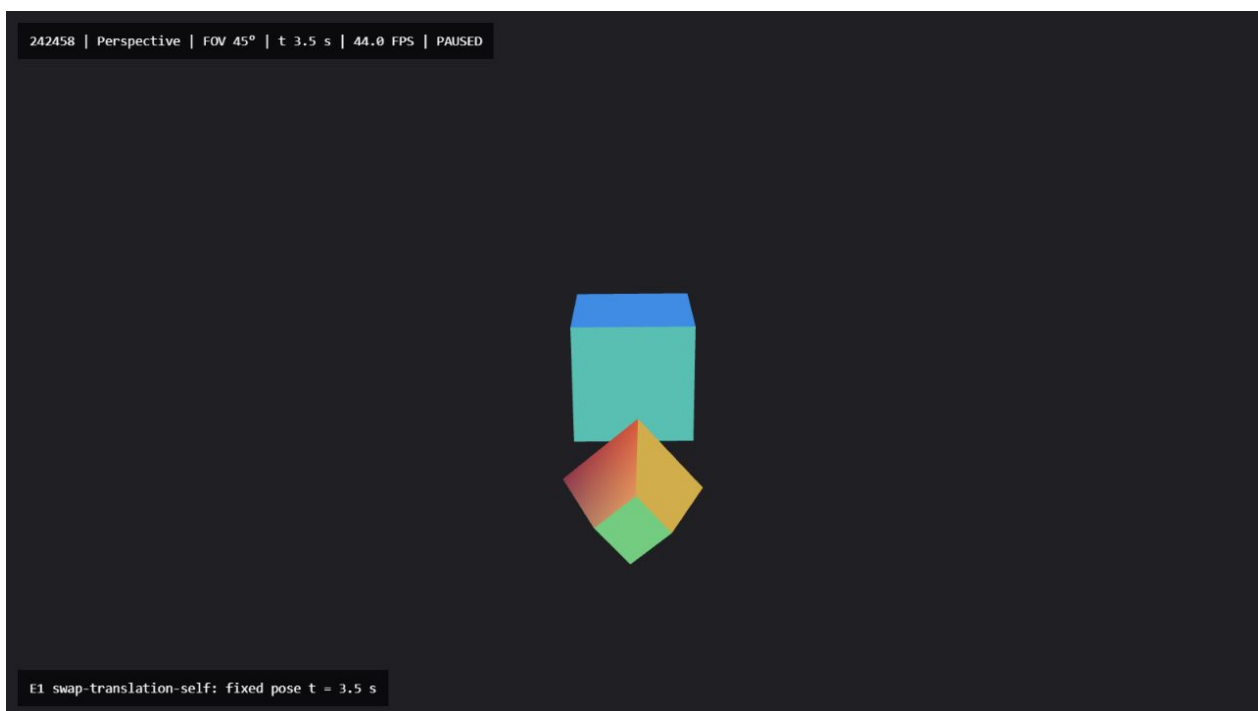


Figure 3. Translation/self-spin swapped: this y-axis variant matches the baseline.

E2 - Three statements in Chapter 4

Prediction: Checking the rotation axis and translating vectors with different w values should clarify the chapter's comments.

Result: (a) [0,1,0] means rotation around y; the comment saying X is incorrect. (b) Cutting geometry at the frustum boundary is clipping. Face culling instead rejects faces based on their orientation. (c) For translation (2,3,4), `vec4.transformMat4` gives $[1,0,0,0] \rightarrow [1,0,0,0]$ and $[1,0,0,1] \rightarrow [3,3,4,1]$ (Figure 4).

Explanation: The axis argument determines the rotation. Clipping can create triangle-boundary intersections; face culling removes whole faces. Here $w=0$ represents a direction and $w=1$ a point, so only the point receives translation. This w is not a quaternion component. After perspective projection, clip-space xyz is divided by clip-space w.

```
242458 | E2 homogeneous coordinates | Console-output evidence

Axis [0,1,0], 90 degrees:
[1,0,0,1] -> [0, 0, -1, 1]

Translation: [2,3,4]
[1,0,0,0] -> [1, 0, 0, 0]
[1,0,0,1] -> [3, 3, 4, 1]
```

Figure 4. E2 translation of a direction and a point, with console results displayed.

E3 - Perspective and orthographic projection

Prediction: At the same paused pose, the nearer vertical cube edge should look longer in perspective. Both edges should have equal lengths in orthographic projection.

Result: At $t=0$, with a 1280x720 viewport and FOV 45° , the near/far lengths are 116.65/104.41 pixels in perspective, giving a ratio of 1.1172. In orthographic projection they are 110.11/110.11 pixels, giving 1.0000 (Figures 5-6). These lengths come from the endpoints projected into screen coordinates using $P \cdot V \cdot M$. A cyan guide marks the hidden far edge.

Explanation: Perspective divides by a w that depends on depth, making farther objects smaller. Orthographic projection keeps $w=1$, so depth does not change the length. PA1's fixed depth offset is closer to orthographic projection: it shifts points without shrinking them with distance. More precisely, it imitates an oblique parallel projection.

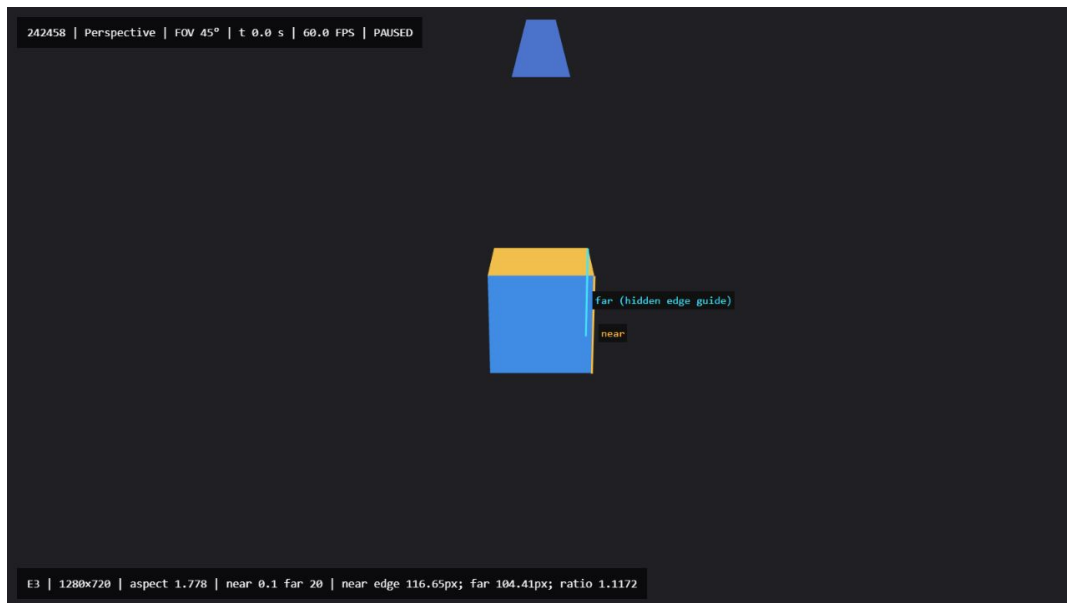


Figure 5. E3 perspective, same paused $t=0$ pose.

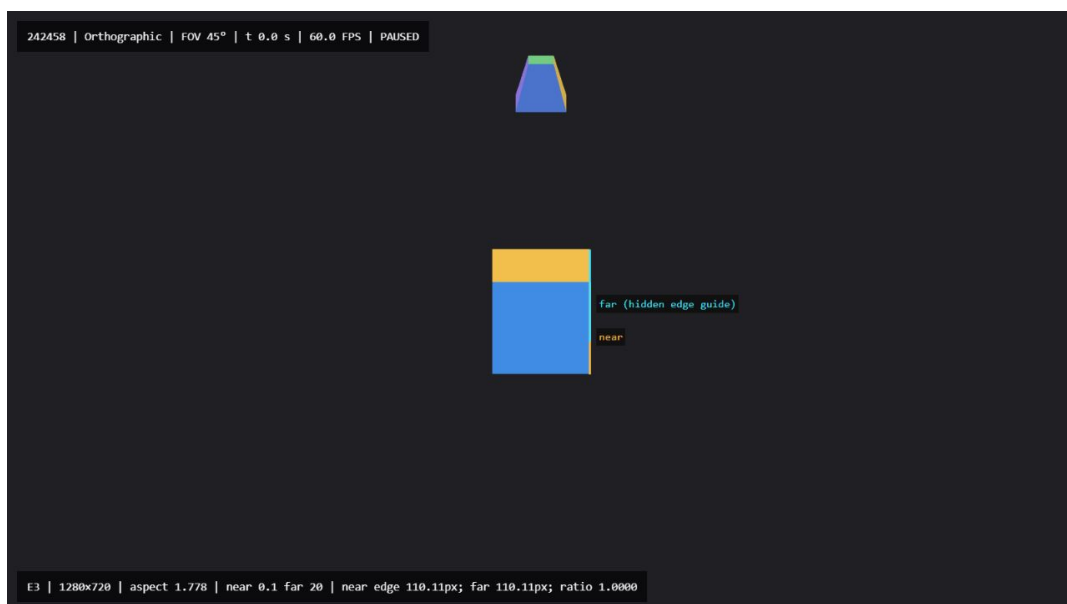


Figure 6. E3 orthographic, same paused $t=0$ pose.

E4 - Near, far and aspect ratio

Prediction: near=0.1 and far=20 should contain the whole orbit. Raising near to 5.2 should cut the solid. Using aspect=1 in a wide window should stretch the scene.

Result: At $t=3.5$ s, near=5.2 cuts the solid's faces, while near=0.1 keeps them visible (Figures 7-8). A 720x720 window looks correct with aspect=1. At 1440x720, aspect=1 stretches the objects horizontally; changing it to 2 restores their proportions (Figures 9-11).

Explanation: The eye is $\sqrt{55.25} \approx 7.433$ units from the origin. The orbit radius is 2.5, and the solid's maximum radius is bounded by $0.8\sqrt{3}/2$. Every vertex therefore lies within 3.193 units of the origin, giving eye-distance bounds of 4.24-10.63 throughout the orbit. These also bound the positive camera-axis depths, so near=0.1 and far=20 safely contain the scene. Sampling the orbit gives tighter depth values of 4.63-10.23. The near plane cuts geometry by camera-axis depth. For a viewport twice as wide as it is tall, aspect=1 makes horizontal dimensions twice their correct size.

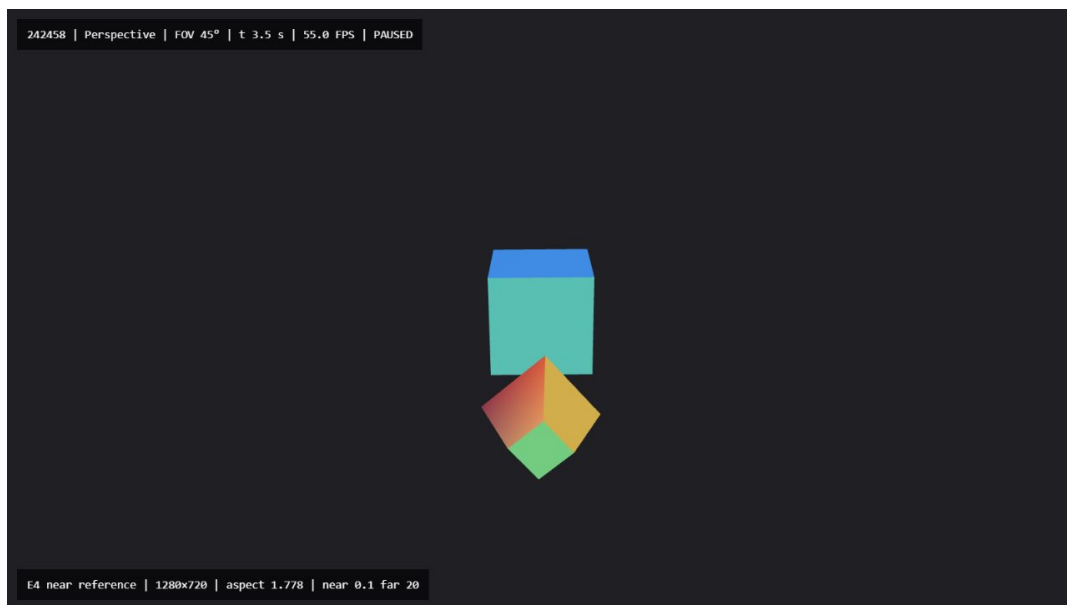


Figure 7. E4 reference near=0.1 at $t=3.5$.

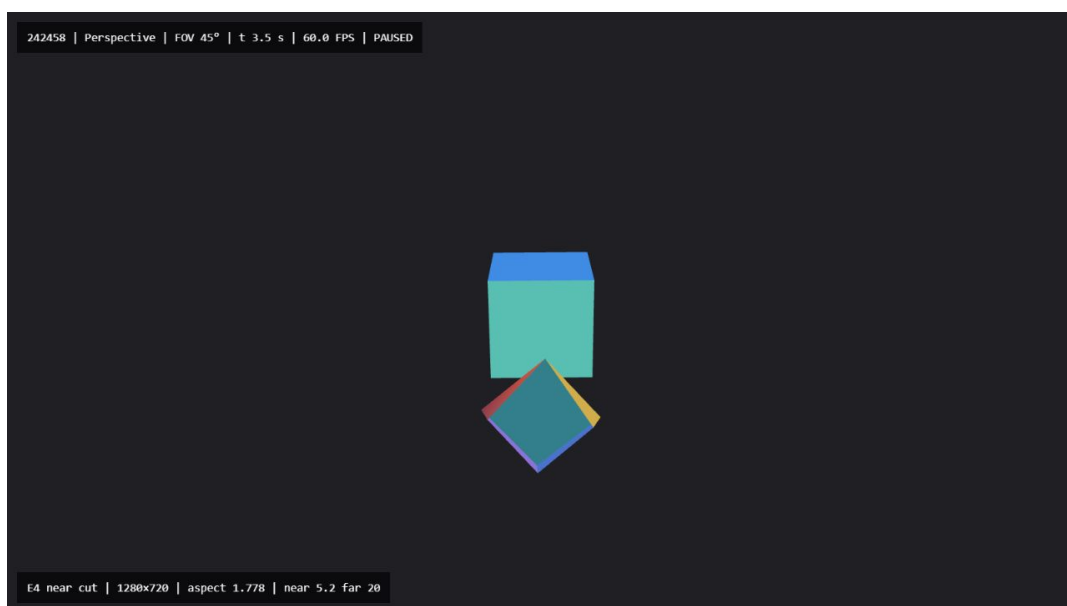


Figure 8. E4 near=5.2 intersects the frustum at the same pose.

E4 - Aspect comparison

All three views below use the same paused pose, $t=0$.

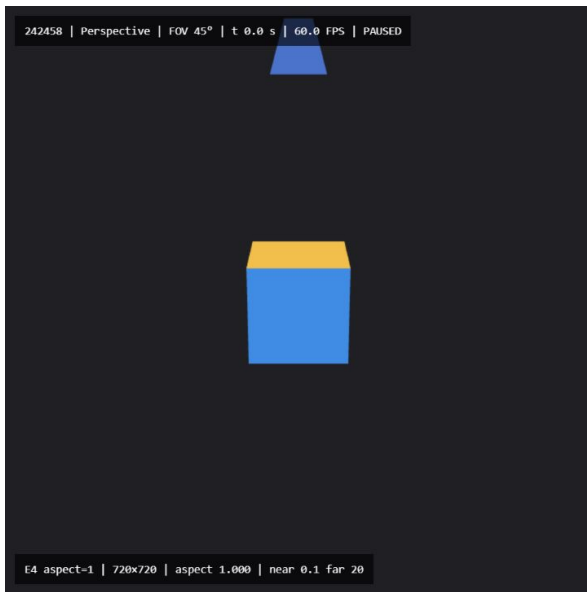


Figure 9. 720x720, aspect=1.



Figure 10. 1440x720, aspect=1: horizontal stretching.

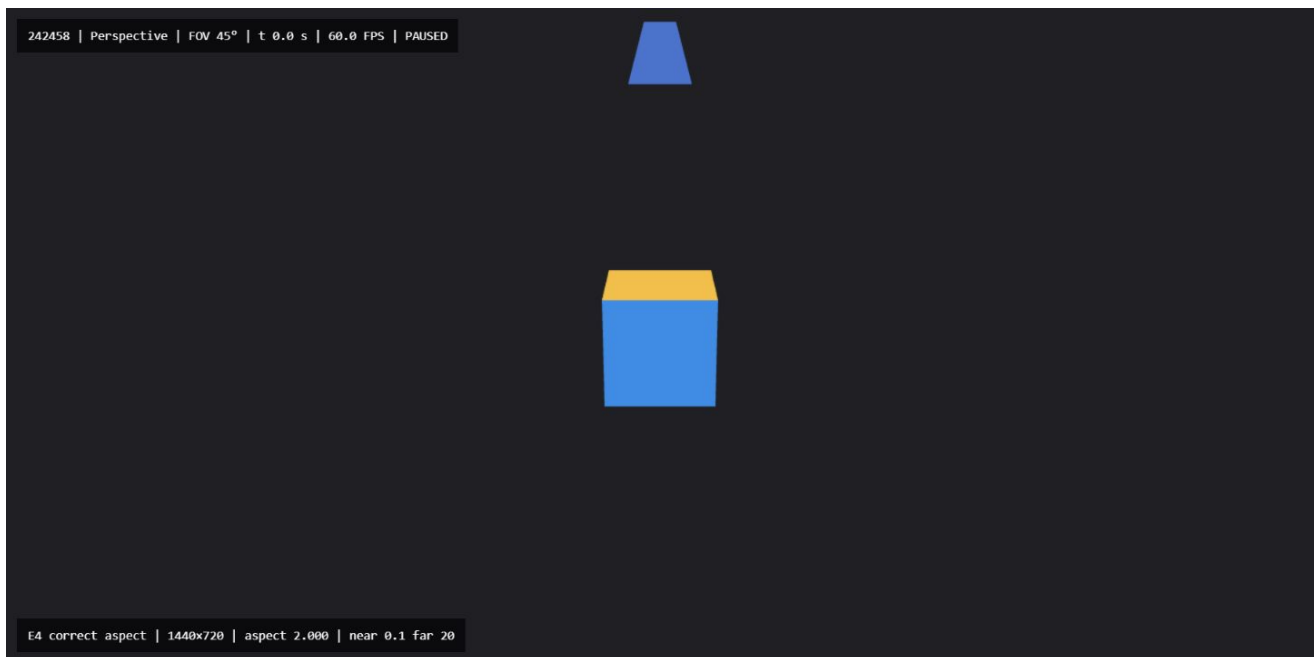


Figure 11. 1440x720, aspect=2: shape preserved.

E5 - Frame-rate independence

Prediction: Using elapsed time should keep the orbit close to $T=14$ s under CPU throttling. Without the dt clamp, returning after five seconds in another tab should cause a jump.

Result: The five-second baseline has 293 intervals, mean $dt=0.01706485$ s and maximum $dt=0.0999$ s. One orbit takes 14.0167 s (Figure 12). With Edge DevTools CPU slowdown set to 4x, the five-second sample has 291 intervals, mean $dt=0.01718214$ s and maximum $dt=0.05$ s. The orbit takes 14.0002 s (Figure 13). Without the clamp, the tab is hidden for 5.1263 s. The next dt is 5.1334 s, advancing t from 88.6986 to 93.8320 and visibly jumping ahead (Figures 14-15).

Explanation: Motion uses elapsed seconds rather than frame count. Both orbit times differ from 14 s by less than one final frame interval. The scene still runs near 60 FPS with 4x slowdown. A hidden tab stops receiving regular animation frames, so the next dt includes the time away. The 0.1 s clamp limits the jump but discards most of a long gap.

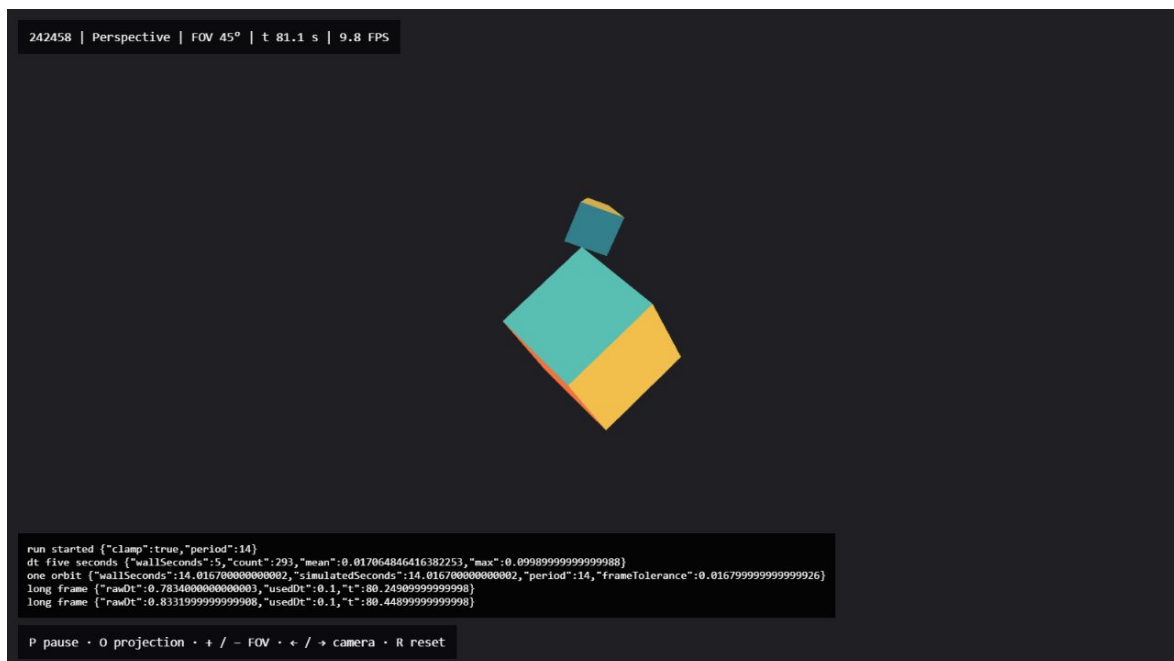


Figure 12. E5 unthrottled baseline and the first orbit results.

E5 - CPU throttling evidence

The screenshot shows the 4x CPU setting and the first sample and orbit. Later suspension logs are not part of those measurements.

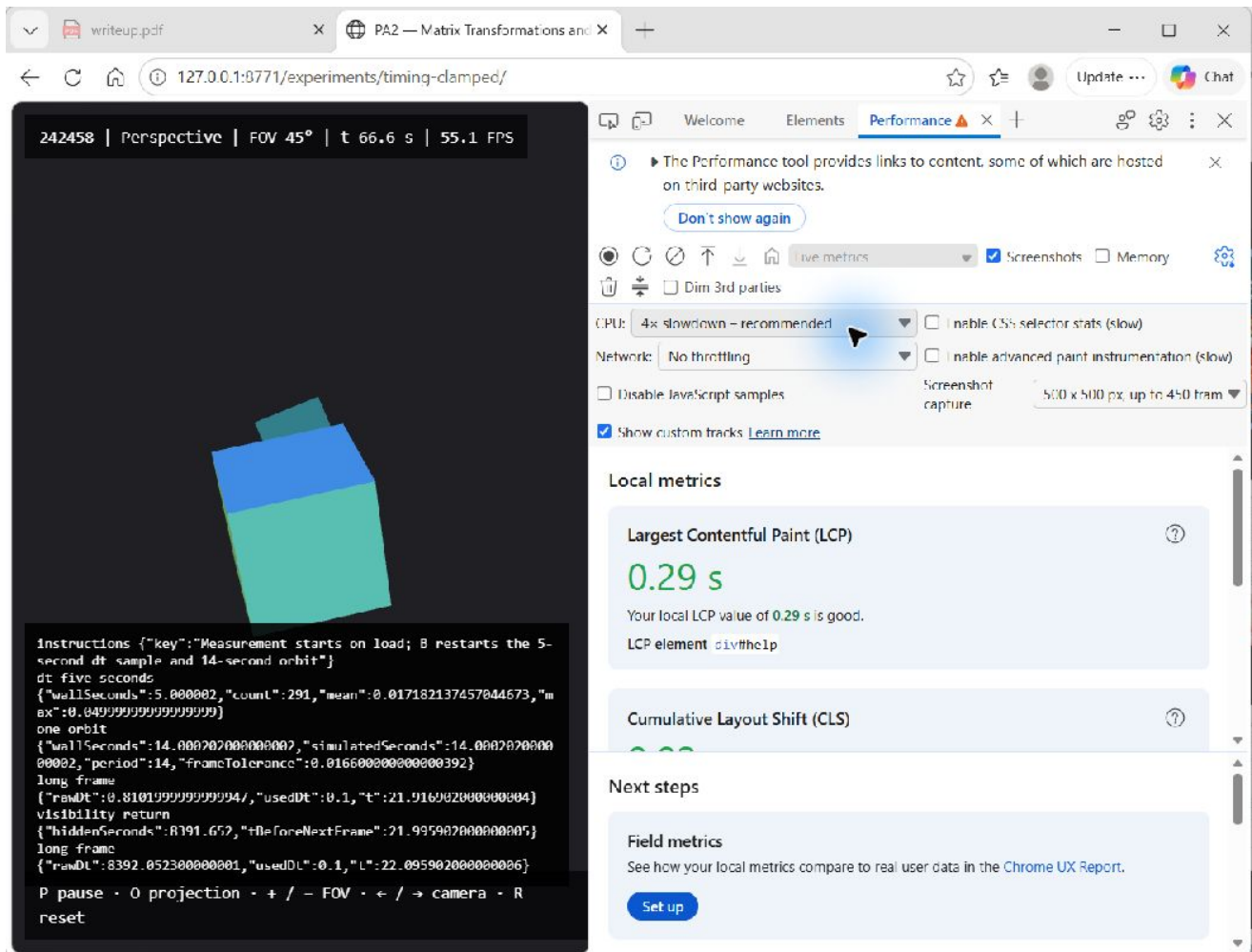


Figure 13. CPU 4x setting, dt sample and orbit time.

E5 - Hidden-tab return without the clamp

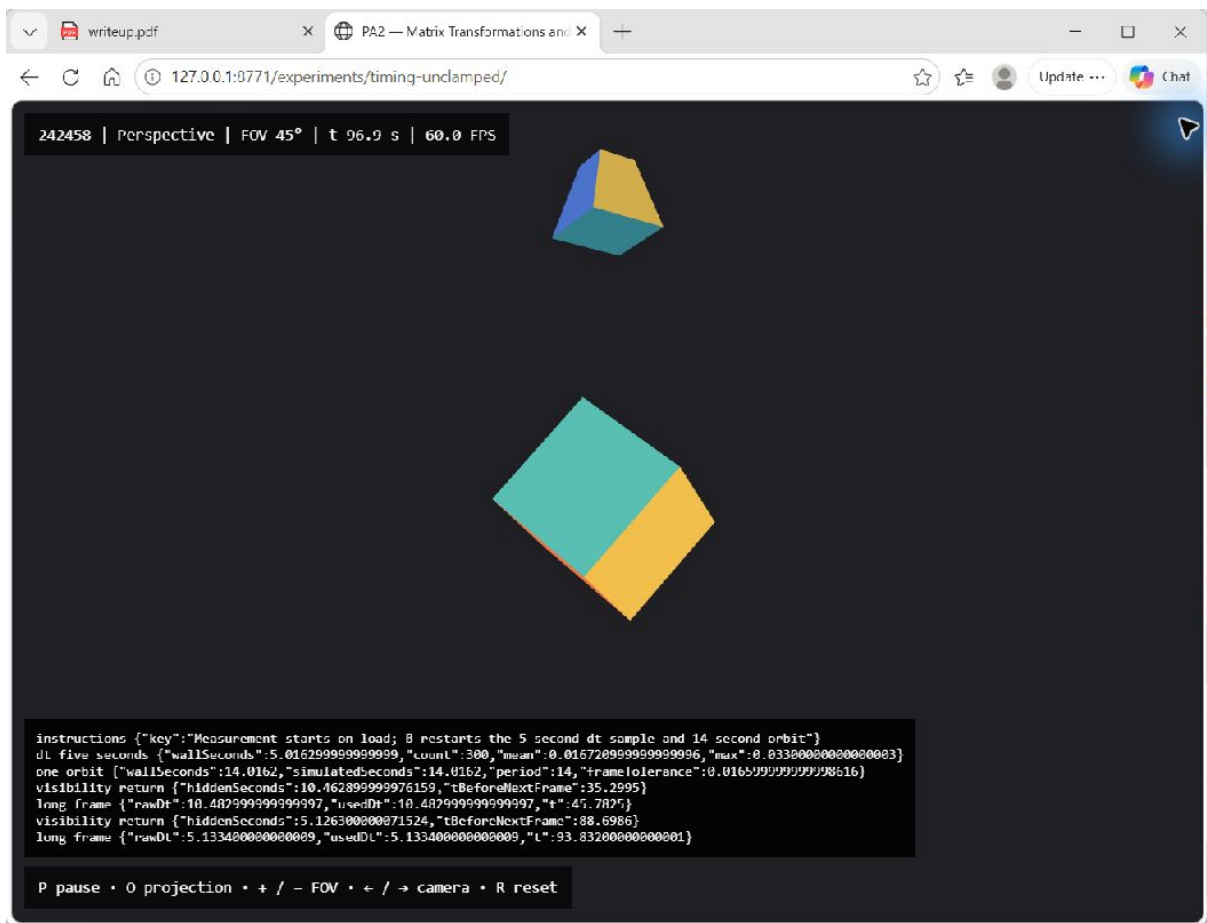


Figure 14. Actual hiding 5.1263 s; first returned dt 5.1334 s.

The console below shows the return interval. CPU throttling was turned off before this tab-switch test.

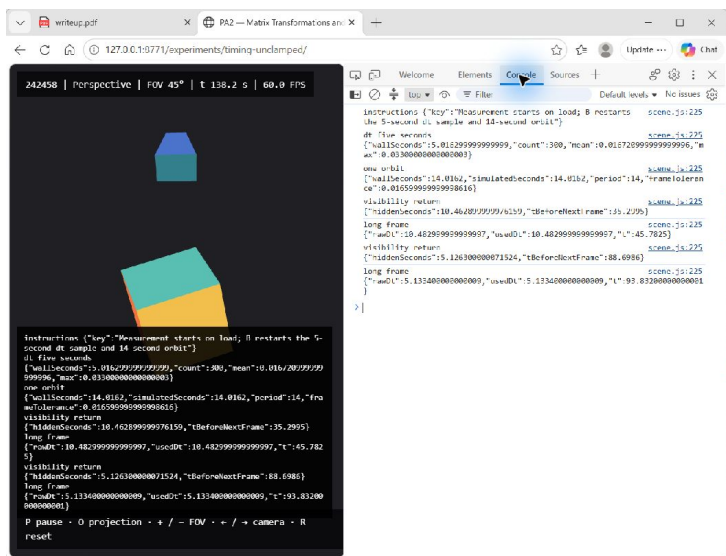


Figure 15. Console evidence of the unclamped return.

E6 - One vertex by hand

Prediction: For the normalised (1,1,1) axis, $t=(2\pi/3)/1.2=1.745329252$ s gives a 120° rotation. The model matrix has rows (0,0,1,0), (1,0,0,0), (0,1,0,0), (0,0,0,1). Thus it maps (x,y,z,w) to (z,x,y,w).

Result: For $v=(0.5,0.5,-0.5,1)$, multiplying by these rows gives $x'=-0.5$, $y'=0.5$, $z'=0.5$ and $w'=1$. `vec4.transformMat4` agrees. The console gives $P \cdot V \cdot M \cdot v = (-0.6789975762, 0.7307890654, 6.6612720490, 6.7939953804)$ (Figure 16). Dividing each of the first three values by 6.7939953804 gives NDC (-0.0999408357, 0.1075639627, 0.9804646126).

Explanation: The model matrix rotates the vertex; view and projection then turn it into clip coordinates. The perspective divide uses the resulting clip w, not the original $w=1$. Small residual values near zero come from floating-point rounding.

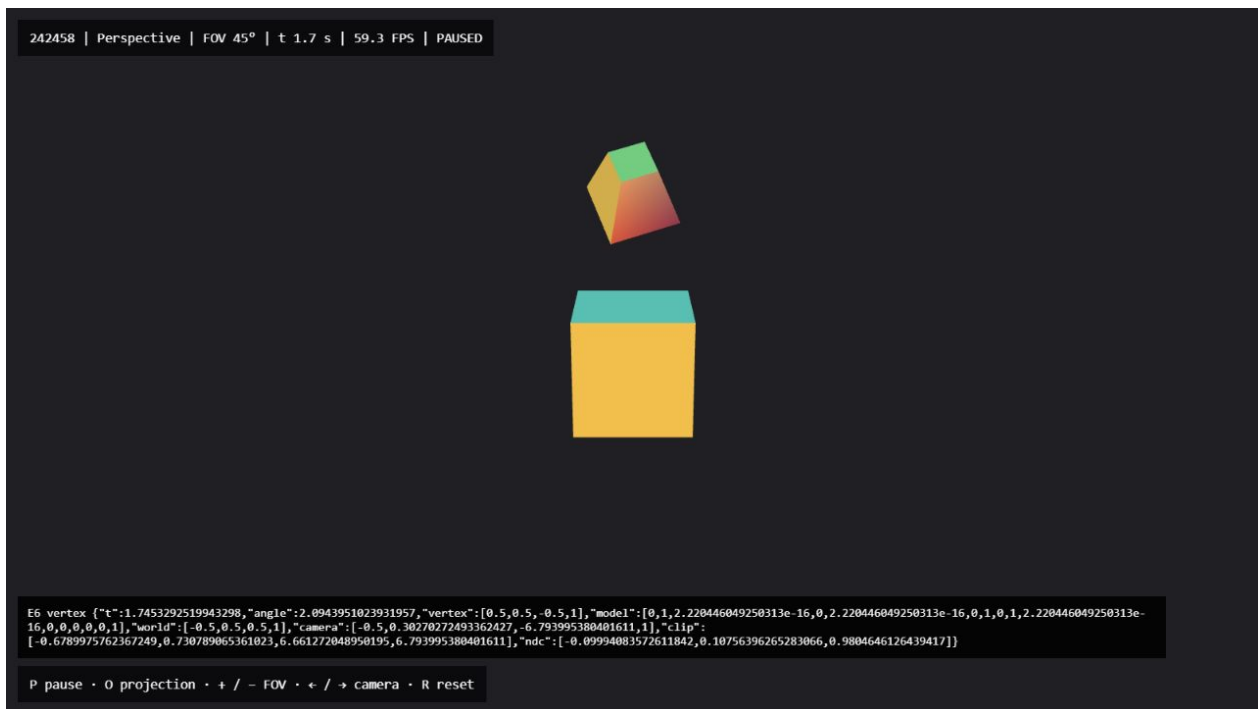


Figure 16. E6 at $t=1.745329252$ s with the world, clip and NDC console result.

Development log

The FPS counter counted the first timestamp even though it had no frame interval. Comparing the frame count with the timestamp intervals revealed the extra count. Commit 390cba5 skips that first count and measures FPS over at least one second.